

# PYTHON PROGRAMMING

## COMPLETE HANDBOOK — BASIC TO ADVANCED

### PART 4

Module 11: Functions

Module 12: Modules and Packages

Module 13: Exception Handling

*Continues from Parts 1-3 (Modules 1-10)*

## TABLE OF CONTENTS

---

|                                                |           |
|------------------------------------------------|-----------|
| <b>TABLE OF CONTENTS</b>                       | <b>2</b>  |
| <b>MODULE 11: FUNCTIONS</b>                    | <b>3</b>  |
| 11.1 User-defined Functions                    | 3         |
| 11.2 Parameters                                | 3         |
| 11.3 Arguments                                 | 3         |
| 11.4 Return Statement                          | 4         |
| 11.5 Lambda Functions                          | 4         |
| 11.6 Recursion                                 | 5         |
| 11.7 Scope of Variables                        | 5         |
| 30+ Function Programs                          | 6         |
| Interview Questions — Module 11                | 8         |
| MCQs — Module 11                               | 9         |
| Exercises — Module 11                          | 9         |
| <b>MODULE 12: MODULES AND PACKAGES</b>         | <b>10</b> |
| 12.1 Importing Modules                         | 10        |
| 12.2 Creating Modules                          | 10        |
| 12.3 Built-in Modules                          | 10        |
| 12.4 Packages                                  | 11        |
| Interview Questions — Module 12                | 11        |
| MCQs — Module 12                               | 12        |
| Exercises — Module 12                          | 12        |
| <b>MODULE 13: EXCEPTION HANDLING</b>           | <b>13</b> |
| 13.1 try                                       | 13        |
| 13.2 except                                    | 13        |
| 13.3 else                                      | 13        |
| 13.4 finally                                   | 14        |
| 13.5 Custom Exceptions                         | 14        |
| Common Error Examples                          | 15        |
| Debugging Tips                                 | 15        |
| Interview Questions — Module 13                | 15        |
| MCQs — Module 13                               | 15        |
| Exercises — Module 13                          | 16        |
| <b>QUICK REVISION NOTES — MODULES 11-13</b>    | <b>17</b> |
| <b>COMMON ERRORS AND FIXES — MODULES 11-13</b> | <b>18</b> |
| <b>CODING BEST PRACTICES — MODULES 11-13</b>   | <b>19</b> |

## MODULE 11: FUNCTIONS

A function is a named, reusable block of code that performs a specific task. Instead of repeating the same code, you write it once inside a function and call it whenever needed — this is central to writing clean, maintainable programs.

### 11.1 User-defined Functions

```
def greet():
    print("Hello, welcome!")

greet()    # calling the function
```

#### Output:

Hello, welcome!

#### Real-World Analogy

A function is like a recipe. You write the recipe once (define it); every time you want the dish, you just follow the recipe (call the function) instead of re-inventing it.

### 11.2 Parameters

Parameters are placeholders listed in a function's definition; they receive the values passed in when the function is called.

```
def greet(name):    # 'name' is a parameter
    print(f"Hello, {name}!")

greet("Aman")
```

#### Output:

Hello, Aman!

### 11.3 Arguments

Arguments are the actual values you pass into a function when calling it. Python supports several argument styles:

| Style             | Meaning                        | Example      | Result            |
|-------------------|--------------------------------|--------------|-------------------|
| <b>Positional</b> | Matched to parameters by order | add(2,3)     | 5                 |
| <b>Keyword</b>    | Matched to parameters by name  | add(a=2,b=3) | 5                 |
| <b>Default</b>    | Used if no value is passed     | def f(x=10): | x=10 unless given |

| Style           | Meaning                                   | Example          | Result           |
|-----------------|-------------------------------------------|------------------|------------------|
| <b>*args</b>    | Collects extra positional args as a tuple | def f(*args):    | args = (1,2,3)   |
| <b>**kwargs</b> | Collects extra keyword args as a dict     | def f(**kwargs): | kwargs = {'a':1} |

```
def total(*args):
    return sum(args)

print(total(1, 2, 3, 4))  # any number of positional args

def show_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

show_info(name="Riya", age=21)
```

**Output:**

```
10
name: Riya
age: 21
```

## 11.4 Return Statement

return sends a value back to wherever the function was called, and immediately ends the function's execution. A function without a return statement returns None by default.

```
def square(n):
    return n * n

result = square(5)
print("Square:", result)
```

**Output:**

```
Square: 25
```

## 11.5 Lambda Functions

A lambda is a small, anonymous (unnamed) function defined in a single line, typically used for short throwaway operations, especially as an argument to functions like sorted() or map().

```
square = lambda x: x * x
print(square(6))

# common use: as a key function for sorting
students = [("Riya", 88), ("Aman", 72)]
students.sort(key=lambda s: s[1])
print(students)
```

**Output:**

```
36
[('Aman', 72), ('Riya', 88)]
```

## 11.6 Recursion

A recursive function is one that calls itself to solve a smaller version of the same problem. Every recursive function needs a base case (a condition to stop) or it will recurse forever and crash with a `RecursionError`.

```
def factorial(n):
    if n == 0:          # base case
        return 1
    return n * factorial(n - 1)  # recursive case

print(factorial(5))
```

Recursion trace for `factorial(5)`:

```
factorial(5) = 5 * factorial(4)
             = 5 * (4 * factorial(3))
             = 5 * (4 * (3 * factorial(2)))
             = 5 * (4 * (3 * (2 * factorial(1))))
             = 5 * (4 * (3 * (2 * (1 * factorial(0)))))
             = 5 * 4 * 3 * 2 * 1 * 1 = 120
```

**Output:**

```
120
```

## 11.7 Scope of Variables

Scope determines where a variable can be accessed from:

- Local scope: a variable created inside a function; only accessible within that function.
- Global scope: a variable created outside all functions; accessible everywhere in the file, including inside functions (for reading).

```
x = 10    # global variable

def show():
    y = 5    # local variable, only exists inside show()
    print(x, y)    # can read the global x here

show()
print(x)    # works - x is global
# print(y)    # NameError - y only exists inside show()
```

### Common Beginner Mistake

Trying to modify a global variable inside a function using just `x = x + 1` creates a NEW local variable instead of changing the global one, causing a confusing `UnboundLocalError`. Use the `'global'` keyword inside the function if you truly need to modify the global variable.

## 30+ Function Programs

A quick-reference set of short function programs — type each one and confirm the output.

| # | Task                  | Code                                                                                                                                                | Output   |
|---|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1 | Add two numbers       | <pre>def add(a,b):     return a+b print(add(3,4))</pre>                                                                                             | 7        |
| 2 | Check even/odd        | <pre>def is_even(n):     return n%2==0 print(is_even(6))</pre>                                                                                      | True     |
| 3 | Find factorial (loop) | <pre>def fact(n):     r=1     for i in range(1,n+1): r*=i     return r print(fact(5))</pre>                                                         | 120      |
| 4 | Reverse a string      | <pre>def rev(s):     return s[::-1] print(rev("abc"))</pre>                                                                                         | cba      |
| 5 | Max of two numbers    | <pre>def mx(a,b):     return a if a&gt;b else b print(mx(4,9))</pre>                                                                                | 9        |
| 6 | Sum of a list         | <pre>def total(lst):     return sum(lst) print(total([1,2,3]))</pre>                                                                                | 6        |
| 7 | Check prime           | <pre>def is_prime(n):     if n&lt;2: return False     for i in range(2,n):         if n%i==0: return False     return True print(is_prime(7))</pre> | True     |
| 8 | Default argument      | <pre>def greet(name='Guest'):     print(f"Hi {name}") greet()</pre>                                                                                 | Hi Guest |
| 9 | Keyword arguments     | <pre>def info(name, age):     print(name, age) info(age=20, name='Sam')</pre>                                                                       | Sam 20   |

| #  | Task                               | Code                                                                                           | Output            |
|----|------------------------------------|------------------------------------------------------------------------------------------------|-------------------|
| 10 | Variable arguments (*args)         | <pre>def total(*a):     return sum(a) print(total(1,2,3,4))</pre>                              | 10                |
| 11 | Keyword variable args (**kwargs)   | <pre>def show(**k):     print(k) show(a=1,b=2)</pre>                                           | {'a': 1, 'b': 2}  |
| 12 | Lambda to square                   | <pre>sq = lambda x: x*x print(sq(5))</pre>                                                     | 25                |
| 13 | Sort with lambda key               | <pre>l=[(1,'b'),(2,'a')] l.sort(key=lambda x:x[1]) print(l)</pre>                              | [(2,'a'),(1,'b')] |
| 14 | Recursive sum 1 to n               | <pre>def s(n):     if n==0: return 0     return n+s(n-1) print(s(5))</pre>                     | 15                |
| 15 | Fibonacci (recursive)              | <pre>def fib(n):     if n&lt;=1: return n     return fib(n-1)+fib(n-2) print(fib(6))</pre>     | 8                 |
| 16 | Function returning multiple values | <pre>def minmax(l):     return min(l), max(l) print(minmax([3,1,9]))</pre>                     | (1, 9)            |
| 17 | Count vowels using function        | <pre>def cv(s):     return sum(1 for c in s if c in "aeiou") print(cv("hello"))</pre>          | 2                 |
| 18 | Check palindrome (function)        | <pre>def pal(s):     return s==s[::-1] print(pal("madam"))</pre>                               | True              |
| 19 | Global keyword usage               | <pre>x=5 def upd():     global x     x+=1 upd(); print(x)</pre>                                | 6                 |
| 20 | Function as argument               | <pre>def apply(f,v):     return f(v) print(apply(lambda x:x+1, 5))</pre>                       | 6                 |
| 21 | Nested function                    | <pre>def outer():     def inner():         return "hi"     return inner() print(outer())</pre> | hi                |
| 22 | Function with no return            | <pre>def f():     pass print(f())</pre>                                                        | None              |
| 23 | Power using recursion              | <pre>def power(b,e):     if e==0: return 1     return b*power(b,e-1) print(power(2,4))</pre>   | 16                |
| 24 | Filter list using function         | <pre>def pos(l):     return [x for x in l if x&gt;0] print(pos([-1,2,-3,4]))</pre>             | [2, 4]            |

| #  | Task                 | Code                                                          | Output    |
|----|----------------------|---------------------------------------------------------------|-----------|
| 25 | map() with lambda    | <pre>l=[1,2,3] print(list(map(lambda x:x*2,l)))</pre>         | [2, 4, 6] |
| 26 | filter() with lambda | <pre>l=[1,2,3,4] print(list(filter(lambda x:x%2==0,l)))</pre> | [2, 4]    |

Full worked example — GCD Using Recursion (with line-by-line):

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

print(gcd(48, 18))
```

| Line | Explanation                                                                                |
|------|--------------------------------------------------------------------------------------------|
| 1    | Define gcd with two parameters a and b.                                                    |
| 2-3  | Base case: when b becomes 0, a itself is the GCD — recursion stops here.                   |
| 4    | Recursive case: uses the Euclidean algorithm — $\text{gcd}(a,b) = \text{gcd}(b, a \% b)$ . |
| 6    | Call and print the result for gcd(48, 18).                                                 |

**Output:**

6

## Interview Questions — Module 11

### Q1. What is the difference between a parameter and an argument?

Ans: A parameter is the placeholder name listed in the function definition (`def f(x):`). An argument is the actual value supplied when the function is called (`f(5)` — 5 is the argument).

### Q2. What is the difference between \*args and \*\*kwargs?

Ans: \*args collects any number of extra positional arguments into a tuple. \*\*kwargs collects any number of extra keyword arguments into a dictionary. Both let a function accept a variable number of inputs.

### Q3. What does a function return if it has no return statement?

Ans: None — every Python function returns something; if return is never reached or never written, the default returned value is None.

### Q4. What is a lambda function and when would you use one?

Ans: A lambda is a small anonymous function written in one line (`lambda args: expression`), typically used for short, throwaway logic passed as an argument to functions like `sorted()`, `map()`, or `filter()`.

### Q5. What is recursion, and what must every recursive function have?

Ans: Recursion is when a function calls itself to solve smaller sub-problems of the same type. Every recursive function must have a base case that stops the recursion, or it will run indefinitely and raise a `RecursionError`.

### Q6. What is variable scope? Explain local vs global.



Ans: Scope defines where a variable is visible. A local variable is defined inside a function and only exists there. A global variable is defined outside any function and can be read anywhere in the file (though modifying it inside a function requires the 'global' keyword).

#### Q7. Why use default parameter values?

Ans: They let a function be called with fewer arguments, falling back to a sensible value automatically, which makes functions more flexible without forcing every caller to specify every argument.

### MCQs — Module 11

| No. | Question                                                           | Answer   | Options (A-D)                                                                        |
|-----|--------------------------------------------------------------------|----------|--------------------------------------------------------------------------------------|
| 1   | A function that calls itself is called:                            | <b>C</b> | A) Iterative B) Nested C) Recursive D) Lambda                                        |
| 2   | *args in a function collects extra arguments as a:                 | <b>B</b> | A) List B) Tuple C) Dictionary D) Set                                                |
| 3   | **kwargs collects extra keyword arguments as a:                    | <b>D</b> | A) List B) Tuple C) Set D) Dictionary                                                |
| 4   | A function with no return statement returns:                       | <b>A</b> | A) None B) 0 C) Empty string D) Error                                                |
| 5   | Lambda functions are also called:                                  | <b>B</b> | A) Named functions B) Anonymous functions C) Recursive functions D) Static functions |
| 6   | Which keyword lets you modify a global variable inside a function? | <b>C</b> | A) local B) nonlocal C) global D) static                                             |
| 7   | Every recursive function must have a:                              | <b>D</b> | A) Loop B) Lambda C) Default arg D) Base case                                        |

### Exercises — Module 11

1. Write a function to check if a number is prime.
2. Write a recursive function to compute the nth Fibonacci number.
3. Write a function using \*args to find the maximum of any number of arguments.
4. Write a function with a default parameter that calculates simple interest.
5. Use a lambda function with sorted() to sort a list of tuples by their second element.
6. Write a function to demonstrate the difference between local and global scope with an example.

## MODULE 12: MODULES AND PACKAGES

As programs grow, it becomes impractical to keep everything in one file. Python lets you split code into modules (single .py files) and organise related modules into packages (folders of modules), then reuse them via import.

### 12.1 Importing Modules

```
import math
print(math.sqrt(16))          # 4.0

from math import pi, sqrt    # import specific names
print(pi)

import math as m              # import with an alias
print(m.pow(2, 3))
```

**Output:**

```
4.0
3.141592653589793
8.0
```

### 12.2 Creating Modules

Any .py file is automatically a module — its functions and variables can be imported into another file.

```
# --- File: mymath.py ---
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

```
# --- File: main.py (in the same folder) ---
import mymath

print(mymath.add(3, 4))
print(mymath.subtract(10, 4))
```

**Output:**

```
7
6
```

### 12.3 Built-in Modules

| Module        | Purpose                                            | Example              |
|---------------|----------------------------------------------------|----------------------|
| <b>math</b>   | Mathematical functions: sqrt, pow, floor, ceil, pi | math.sqrt(9)         |
| <b>random</b> | Generate random numbers/choices                    | random.randint(1,10) |

| Module          | Purpose                                     | Example                     |
|-----------------|---------------------------------------------|-----------------------------|
| <b>datetime</b> | Work with dates and times                   | <code>datetime.now()</code> |
| <b>os</b>       | Interact with the operating system/files    | <code>os.getcwd()</code>    |
| <b>sys</b>      | Interact with the Python interpreter itself | <code>sys.exit()</code>     |
| <b>json</b>     | Encode/decode JSON data                     | <code>json.dumps(d)</code>  |

## 12.4 Packages

A package is a folder containing multiple related modules, plus a special (often empty) `__init__.py` file that tells Python to treat the folder as a package.

```
myapp/
  __init__.py
  calculator/
    __init__.py
    basic.py      # def add(), subtract()...
    advanced.py   # def power(), sqrt()...
  main.py

# Usage inside main.py:
from calculator import basic
print(basic.add(2, 3))
```

### Real-World Analogy

A module is like one chapter of a book. A package is the whole book, organising many related chapters (modules) under one cover (folder), so you can import exactly the chapter you need.

## Interview Questions — Module 12

### Q1. What is the difference between a module and a package?

Ans: A module is a single `.py` file containing functions, classes, or variables. A package is a folder containing multiple related modules, identified by an `__init__.py` file, used to organise larger codebases.

### Q2. What is the difference between `import module` and `from module import name`?

Ans: `import module` brings in the whole module, and its contents must be accessed with the module prefix (`module.name`). `from module import name` imports a specific name directly, so it can be used without the prefix.

### Q3. What does `'import math as m'` do?

Ans: It imports the `math` module but binds it to the shorter alias `m`, so you can write `m.sqrt()` instead of `math.sqrt()` — useful for long or frequently-used module names.

### Q4. Name three commonly used built-in Python modules.

Ans: `math` (mathematical functions), `random` (random number generation), `datetime` (date/time handling) — also `os`, `sys`, `json` are common.

### Q5. What is the purpose of `__init__.py`?

Ans: It marks a folder as a Python package, allowing its modules to be imported using dotted package notation. It can also run initialization code for the package.

**MCQs — Module 12**

| No. | Question                                         | Answer   | Options (A-D)                                                         |
|-----|--------------------------------------------------|----------|-----------------------------------------------------------------------|
| 1   | Which module provides sqrt() and pi?             | <b>B</b> | A) random B) math C) os D) sys                                        |
| 2   | Which statement imports only pi from math?       | <b>C</b> | A) import math.pi B) import pi C) from math import pi D) get math.pi  |
| 3   | A package is identified by the presence of:      | <b>A</b> | A) __init__.py B) main.py C) setup.py D) package.py                   |
| 4   | Which module is used to generate random numbers? | <b>D</b> | A) math B) sys C) os D) random                                        |
| 5   | import math as m is an example of:               | <b>C</b> | A) Package creation B) Selective import C) Import with alias D) Error |

**Exercises — Module 12**

1. Create your own module with two functions and import it into another file.
2. Use the random module to simulate rolling a dice 5 times.
3. Use the datetime module to print today's date and the current time.
4. Use the math module to compute the area of a circle given its radius.

## MODULE 13: EXCEPTION HANDLING

---

An exception is an error that occurs during program execution and, if unhandled, stops the program immediately. Exception handling lets a program detect errors and respond gracefully instead of crashing.

### 13.1 try

Code that might raise an error is placed inside a try block — Python attempts to run it and watches for exceptions.

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

**Output:**

Cannot divide by zero

### 13.2 except

Catches and handles a specific type of exception. You can have multiple except blocks to handle different error types differently.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("That is not a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

**Output:**

Enter a number: abc  
That is not a valid number

### 13.3 else

The else block runs only if the try block completed successfully with NO exception raised.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error!")
else:
    print("Success:", result)
```

**Output:**

Success: 5.0

### 13.4 finally

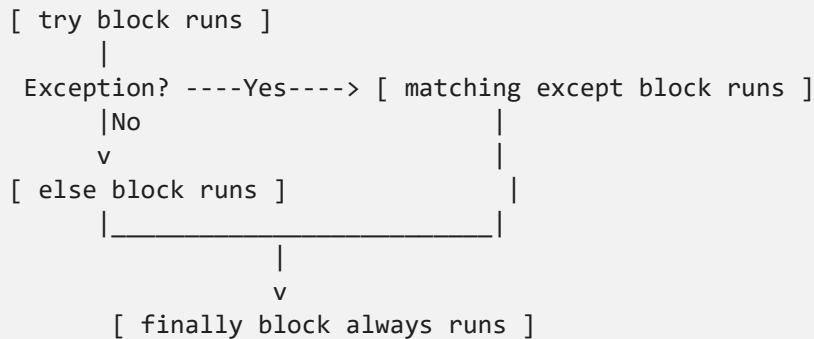
The finally block always runs, whether an exception occurred or not — commonly used to release resources like closing a file or a database connection.

```
try:
    f = open("data.txt")
    # ... work with file ...
except FileNotFoundError:
    print("File not found")
finally:
    print("This always runs – good place to clean up")
```

**Output:**

```
File not found
This always runs – good place to clean up
```

Full try-except-else-finally flow:



### 13.5 Custom Exceptions

You can define your own exception types by creating a class that inherits from Python's built-in Exception class — useful for representing errors specific to your application's logic.

```
class InsufficientBalanceError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError("Not enough balance!")
    return balance - amount

try:
    withdraw(500, 1000)
except InsufficientBalanceError as e:
    print("Error:", e)
```

**Output:**

```
Error: Not enough balance!
```

## Common Error Examples

| Exception                | Typical Cause                       | Example              |
|--------------------------|-------------------------------------|----------------------|
| <b>ZeroDivisionError</b> | Dividing by zero                    | 10 / 0               |
| <b>ValueError</b>        | Wrong value type for conversion     | int("abc")           |
| <b>TypeError</b>         | Operation on incompatible types     | "5" + 5              |
| <b>IndexError</b>        | Index beyond sequence length        | [1,2][5]             |
| <b>KeyError</b>          | Dictionary key doesn't exist        | {'a':1}['b']         |
| <b>FileNotFoundError</b> | Opening a file that doesn't exist   | open("x.txt")        |
| <b>NameError</b>         | Using a variable before defining it | print(undefined_var) |

## Debugging Tips

- Read the LAST line of the traceback first — it names the exact exception type and message.
- Note the line number in the traceback — that's exactly where the error occurred.
- Never use a bare except: (catches everything, including typos) — always catch specific exception types.
- Use print() statements or a debugger to inspect variable values right before the failing line.
- Test edge cases deliberately: empty input, zero, negative numbers, missing files.

## Interview Questions — Module 13

### Q1. What is the difference between an error and an exception?

Ans: An error (like a SyntaxError) usually means the code is structurally invalid and cannot even run. An exception is a runtime problem that occurs while a syntactically valid program is executing (like dividing by zero) and can be caught and handled.

### Q2. What is the purpose of the finally block?

Ans: It always executes, whether or not an exception occurred, and is used for cleanup actions like closing files or releasing resources that must happen regardless of success or failure.

### Q3. When does the else block in a try-except run?

Ans: Only if the try block completes with no exception raised at all — if an exception occurs, else is skipped.

### Q4. How do you create a custom exception in Python?

Ans: By defining a new class that inherits from the built-in Exception class (or one of its subclasses), then raising it with the raise keyword when your specific error condition occurs.

### Q5. Why is using a bare except: considered bad practice?

Ans: It catches every possible exception, including ones you didn't anticipate (like KeyboardInterrupt or programming typos turned into NameError), which can hide real bugs and make debugging much harder.

### Q6. What does the raise keyword do?

Ans: It manually triggers an exception, either a built-in one or a custom one, interrupting normal program flow so it can be caught by an except block.

## MCQs — Module 13

| No. | Question | Answer | Options (A-D) |
|-----|----------|--------|---------------|
|-----|----------|--------|---------------|

| No. | Question                                          | Answer | Options (A-D)                                                |
|-----|---------------------------------------------------|--------|--------------------------------------------------------------|
| 1   | Which block always executes, error or not?        | D      | A) try B) except C) else D) finally                          |
| 2   | Dividing a number by zero raises:                 | B      | A) ValueError B) ZeroDivisionError C) TypeError D) NameError |
| 3   | Which block runs only if NO exception occurred?   | C      | A) try B) except C) else D) finally                          |
| 4   | Custom exceptions are created by inheriting from: | A      | A) Exception B) BaseError C) TryError D) object only         |
| 5   | int("abc") raises:                                | B      | A) TypeError B) ValueError C) NameError D) SyntaxError       |
| 6   | Which keyword manually triggers an exception?     | C      | A) throw B) except C) raise D) error                         |
| 7   | Accessing a missing dictionary key raises:        | D      | A) IndexError B) ValueError C) TypeError D) KeyError         |

### Exercises — Module 13

1. Write a program that handles both ValueError and ZeroDivisionError for a division calculator.
2. Write a program using try-except-finally to safely open and read a file that may not exist.
3. Create a custom exception NegativeAgeError and raise it when a negative age is entered.
4. Write a program that keeps asking for input until a valid integer is entered, using a try-except loop.
5. Explain, with an example, why a bare except: is discouraged.



## QUICK REVISION NOTES — MODULES 11-13

---

- A function is defined with `def`, called by `name()`, and can accept parameters and return a value with `return` (default: `None`).
- `*args` gathers extra positional arguments as a tuple; `**kwargs` gathers extra keyword arguments as a dict.
- `lambda args: expr` creates a small anonymous function, often used inline with `sorted()/map()/filter()`.
- Every recursive function needs a base case to avoid infinite recursion (`RecursionError`).
- Local variables live only inside their function; global variables are visible everywhere but need the `'global'` keyword to be modified inside a function.
- `import module`, `from module import name`, and `import module as alias` are the three main import styles.
- A package is a folder of modules marked with `__init__.py`.
- `try/except/else/finally`: `try` = risky code, `except` = handle error, `else` = runs only if no error, `finally` = always runs.
- Custom exceptions are created by subclassing `Exception` and triggered with `raise`.

## COMMON ERRORS AND FIXES — MODULES 11-13

---

| Error                                            | Cause                                                | Fix                                                 |
|--------------------------------------------------|------------------------------------------------------|-----------------------------------------------------|
| TypeError: missing required positional argument  | Calling a function without enough arguments          | Pass all required arguments, or add default values  |
| RecursionError: maximum recursion depth exceeded | Missing or unreachable base case                     | Ensure the base case is correct and reachable       |
| ModuleNotFoundError                              | Module name misspelled or not installed              | Check spelling; pip install the package if external |
| UnboundLocalError                                | Modifying a global variable without 'global' keyword | Add 'global varname' inside the function            |
| Bare except hides real bugs                      | Using except: with no exception type                 | Always specify the exact exception class            |

## CODING BEST PRACTICES — MODULES 11-13

---

- Keep functions small and focused on a single task — if a function does too much, split it.
- Use descriptive function names that state what they do: `calculate_tax()`, not `ct()`.
- Always add a docstring to non-trivial functions explaining what they do, their parameters, and return value.
- Catch specific exceptions, never a bare `except`: — this prevents hiding real bugs.
- Use `finally` (or a `with` statement) to guarantee resources like files are properly closed.
- Prefer built-in modules over writing your own logic when one already exists (e.g., use `math`, don't hand-roll `sqrt`).